多賣家電商平台 測試計畫

> 狀態：定版
> 版本：1.1
> 最後更新：2026-10-15
> 依據：《需求_001_需求規格》1.8、《需求_002_使用者故事》1.6、《設計_002_狀態圖與循序圖》1.1、《設計_003_系統架構》1.3、《設計_004_資料庫設計》1.2、《設計_005_API規格》1.1、ADR_001、ADR_005、ADR_007、ADR_009、ADR_011、ADR_012、ADR_013

1. 目的與範圍

- 定義測試的層級、各層測什麼、用什麼工具
- 把使用者故事的驗收條件對應到具體的測試
- 定義一個功能「完成」的標準

第 5 節的對應表只涵蓋第一版範圍內的使用者故事；其他故事在各自的階段補上。壓測另寫《壓測計畫》，在併發正確性階段開始前撰寫。

2. 測試的層級

採用測試金字塔：越下層數量越多、執行越快；越上層數量越少、越接近真實使用。

層級	測什麼	範圍	外部依賴	數量
單元測試	純業務邏輯：狀態轉換規則、金額計算、折扣分攤的尾數、冪等雜湊、期限計算	單一類別	無，不啟動 Spring	最多
模組測試	單一模組的完整行為：資料存取、SQL、資料庫限制、交易、發布的事件	單一模組	真的 PostgreSQL；其他模組以替身取代	多
API 測試	驗收條件：透過 HTTP 呼叫 API，驗證回應、資料庫狀態與事件	整個主應用程式	真的 PostgreSQL；模擬金流以 HTTP 替身取代	中
併發測試	同時發生的請求下，資料庫限制與原子操作是否守得住	整個主應用程式或單一模組	真的 PostgreSQL	少，但不可省略
端對端測試	主應用程式與真的模擬金流一起運作	兩個服務	Docker Compose	最少
架構測試	模組邊界、程式慣例	整個程式碼	無	少

2.1 為什麼模組測試與 API 測試都用真的 PostgreSQL

我們有多條規則由資料庫保證：部分唯一索引（待付款上限、處理中付款上限）、`CHECK` 限制（庫存不為負）、原子條件更新、唯一性約束提供的冪等。記憶體資料庫無法忠實重現這些行為，以它測試等於沒

測。

2.2 為什麼 API 測試用模擬金流的替身

API 測試需要精確控制金流的每一種回應，包括延遲、逾時、錯誤格式，且必須快速、穩定。以 HTTP 替身（例如 WireMock）在測試中直接設定回應。

模擬金流本身的行為，以及兩個服務之間的實際串接，由端對端測試驗證。

3. 工具

用途	工具
測試框架	JUnit
斷言	AssertJ
替身	Mockito（單元測試）、WireMock（HTTP）
資料庫	Testcontainers 的 PostgreSQL，版本與正式環境相同
模組測試與模組邊界驗證	Spring Modulith 的測試支援
程式慣例檢查	ArchUnit
API 呼叫	Spring 的 MockMvc 或 RestTestClient
非同步結果的等待	Awaitility；模組測試也可使用 Spring Modulith 的 `Scenario`
回應是否符合規格	以 OpenAPI 規格檔驗證 API 測試的請求與回應，工具於實作時選定
覆蓋率	JaCoCo
執行	單元、模組、架構測試由 Surefire 執行；API、併發、端對端測試由 Failsafe 執行，分工方式見 4.3

4. 共通做法

4.1 時間

- 程式一律透過注入的時鐘取得時間（《設計_003_系統架構》第 6 節）
- 測試使用可以手動前進的時鐘，例如「前進 31 分鐘後執行逾時取消任務」，不使用真實的等待
- 驗證存取權杖期限的元件也使用注入的時鐘（例如 Spring Security 的 `JwtTimestampValidator.setClock`），否則「15 分鐘後失效」無法以可控時鐘測試
- 架構測試禁止在業務程式中直接取得系統時間：`Instant.now()`、`LocalDateTime.now()` 等無參數的 `now()`、`System.currentTimeMillis()`、`Clock.systemUTC()`、`Clock.systemDefaultZone()`、`new Date()`
- SQL 不使用資料庫的 `now()`、`CURRENT_TIMESTAMP`（《設計_004_資料庫設計》12.1）。ArchUnit 檢查不到 SQL，另以測試掃描程式中的 SQL 與 Flyway 腳本；Spring Modulith 事件持久化的資料表由函式庫定義，不在檢查範圍內

4.2 測試資料

- 每個測試自己建立需要的資料，使用新產生的識別碼，不依賴其他測試留下的資料，也不依賴執行順序
- 以建構器建立測試資料，例如「一個有 10 件庫存的上架商品」，只寫出與該測試有關的欄位
- 整個測試執行期間共用一個 PostgreSQL 容器，以縮短時間；因為每個測試的資料互不重疊，不需要在測試之間清空資料庫
- 需要統計全域數量的測試（例如批次任務），以自己建立的資料範圍為條件查詢，不計算全表
- 查詢類的測試（例如搜尋、排序、分頁）以自己建立的分類或獨特的關鍵字限定範圍，否則結果會混入其他測試建立的商品
- 會前進時鐘或執行批次任務的測試，不與其他測試平行執行。例如前進 31 分鐘後執行逾時取消任務，會連帶取消其他測試尚未付款的結帳單
- 時鐘只往前推進，不往回撥，避免之後的測試讀到「時間在未來」的資料。測試以相對於目前時鐘的時間建立資料與斷言，不寫死日期

4.3 命名與可追溯性

- 測試類別以被測對象命名，後綴表示層級，Surefire 與 Failsafe 依後綴分工：

層級	後綴	例子	執行
單元	`Test`	`DiscountAllocationTest`	Surefire
模組	`ModuleTest`	`StockReservationModuleTest`	Surefire
架構	`ArchitectureTest`	`ModuleBoundaryArchitectureTest`	Surefire
API	`ApiIT`	`CheckoutApiIT`	Failsafe
併發	`ConcurrencyIT`	`CheckoutConcurrencyIT`	Failsafe
端對端	`E2EIT`	`PaymentFlowE2EIT`	Failsafe，只在 `e2e` 設定檔中執行

- Failsafe 預設只執行 `*IT` 結尾的類別，Surefire 不執行它們，因此後綴不可省略
- 每個對應驗收條件的測試，以 `@DisplayName` 寫出故事編號與情境，例如 `O3 同時送出：只有一個請求建立結帳單`
- 從測試報告就能看出每個故事的每個情境是否通過

4.4 驗證的內容

API 測試除了 HTTP 回應，還要驗證：

項目	例子
資料庫狀態	庫存數字、結帳單狀態
異動與稽核紀錄	有對應的庫存異動紀錄、狀態歷程
發布的事件	發布了「結帳單已建立」事件
不該發生的事	失敗時沒有建立任何結帳單、沒有預扣庫存

最後一項特別重要：多數錯誤不是「沒做該做的事」，而是「做了不該做的事」。

4.5 非同步事件與排程

- 測試環境關閉所有排程，批次任務由測試直接呼叫。否則排程會在背景執行，與 4.2「不與其他測試平行執行」的前提衝突
- 經由 Outbox 非同步完成的結果，例如結帳後購物車移除商品、付款成功後結帳單變為已付款、帳本分錄，以等待直到條件成立的方式驗證，並設定等待上限，不使用固定時間的休眠
- 需求訂有延遲上限的，以該上限作為等待上限，例如賣家在 5 秒內查到剛付款的子訂單（使用者故事 Q4）
- 測試環境將平台呼叫金流的回應逾時調短（例如 500 毫秒），避免每個逾時情境都等待 3 秒

5. 驗收條件與測試的對應（第一版）

5.1 訂單

故事	情境	層級
O1 結帳並拆單	主流程	API
	折扣分攤與金額計算的規則	單元
	預扣的 SQL 與異動紀錄	模組
O2 結帳失敗	庫存不足、商品已下架、數量超過上限、已有待付款	API
	同時送出兩筆不同的結帳，只有一筆成功	併發
O3 重複送出結帳	先後重送	API
	同時送出	併發
	相同冪等鍵、不同內容	API
O4 逾時取消	沒有處理中的付款、有處理中的付款、處理中的付款後來失敗、任務尚未處理到時發起付款	API（以可控時鐘前進時間）
	付款已成功但訂單尚未更新	API（暫停事件發送，見 6.3）
O6 會員主動取消	可以取消、付款處理中、已付款	API
	付款已成功但訂單尚未更新時，取消被拒絕	API（暫停事件發送，見 6.3）
	商品放回購物車	API
	放回時購物車已有同一項商品	API

- O3 的同時送出：另一個請求回應 200 與同一張結帳單，或 409 `IDEMPOTENCY_KEY_IN_USE`（《設計_005_API規格》2.6），測試接受這兩種回應，但另一個請求不可以回應

`PENDING_CHECKOUT_EXISTS`：相同冪等鍵的兩筆寫入會同時違反冪等與待付款上限兩個唯一索引，實作必須依冪等處理

- O6 放回時已有同一項商品：數量合併且可以超過 99 件，購物車標示超過上限，結帳時回應 `QUANTITY_LIMIT_EXCEEDED`，改為 99 件以下後即可結帳（需求 3.1）

O5（自動簽收與完成）不在第一版範圍。

5.2 付款

故事	情境	層級
P1 付款成功	結帳單與子訂單變為已付款、確認扣除庫存、帳本分錄	API
P2 金流重複回報	同一事件編號送達兩次，只處理一次	API
	同一事件編號同時送達	併發
P3 付款結果不明	金流逾時，付款維持處理中；查詢任務取得成功或失敗	API
	處理中時不能再發起付款	API、併發
	處理中超過 1 小時仍無法確認時發出告警	API（以可控時鐘前進時間）
P4 付款成功但結帳單已取消	訂單模組收到「付款已成功」事件時結帳單已取消：結帳單維持已取消，發布「要求自動退款」事件	模組
	取消與發起付款同時發生，之後付款成功	併發（見 6.1）
P5 付款失敗後重新付款	第二次付款成功；預扣的庫存不變；只有一筆成功的付款	API

P6、P7（退款）不在第一版範圍。

P4 的說明：

- 付款處理中的結帳單不會被取消（O4、O6），所以 P4 在第一版只會因為「取消」與「發起付款」同時發生而出現：取消已確認沒有付款，但在提交之前，另一個請求建立了付款。這種時機無法在 API 測試中穩定重現，因此訂單模組的處理以模組測試直接送入事件驗證，整體結果以併發測試驗證
- 第一版的模擬金流還沒有退款的介面，付款模組收到「要求自動退款」事件時只發出告警（《設計_005_API規格》4.1）。實際向金流退款的部分在退款階段補上測試
- 告警的機制在可觀測性階段建立；第一版的測試驗證寫出告警等級的日誌

5.3 查詢

故事	情境	層級
Q1 搜尋商品	篩選、排序、分頁、超過 1,000 筆被拒絕、已下架不出現	API
Q2 商品頁	庫存狀態依門檻顯示、改價後顯示新價格、以新價格結帳	API
Q3 查看我的訂單	剛下單立即可見、依狀態篩選、他人訂單回應查無資料	API
	訂單詳情：商品快照、付款紀錄、狀態變更歷程	API

故事	情境	層級
Q4 賣家查詢子訂單	篩選、分頁、日期區間、不含其他賣家的子訂單	API
	會員剛付款的子訂單，5 秒內出現在列表中	API
Q5 查詢付款結果	一般查詢、狀態剛改變、他人的結帳單	API

- 第一版只有待付款、已付款、已取消三種狀態，Q3 的依狀態篩選以這三種狀態驗證；Q3 訂單詳情的折扣分攤與退款紀錄在對應的階段補上
- Q5 的尖峰查詢屬於壓測

5.4 賣家、規則告知、認證與權限

故事	情境	層級
S1 資料隔離	查詢、修改他人的商品與訂單	API
N1 條款同意	註冊、未同意、條款更新後須重新同意、舊版保留	API
	管理者同意賣家條款之前，該賣家的所有帳號都無法使用賣家功能；一般成員不能代表賣家同意	API
N2 提醒與系統一致	回應的付款期限與設定值相同	API
A1 登入失敗鎖定	連續失敗、鎖定期間、鎖定結束、失敗後成功	API（以可控時鐘前進時間）
A2 營運角色	依權限表逐格驗證，範圍見下方	API
	角色被移除後，最晚 15 分鐘失去權限	API（以可控時鐘前進時間）
A3 賣家角色	依權限表逐格驗證，範圍見下方	API

權限表以參數化測試逐格驗證，確保表格中每一格都有測試。第一版只驗證已有 API 的格子，其餘在對應的 API 加入時補上：

故事	第一版驗證的範圍
A2	查詢訂單與賣家：四個角色皆可。建立賣家、設定抽成比例、新增分類、發布條款、管理營運帳號與角色：只有營運管理者可以，其他三個角色被拒絕
A2 角色被移除	同時具有營運管理者與財務營運角色的帳號，被移除營運管理者角色後，最晚 15 分鐘無法再設定抽成比例，且更新權杖已被撤銷
A3	商品、庫存、子訂單的查詢與操作：管理者與一般成員皆可。同意賣家條款：只有管理者可以

A2 的權責分離、多個角色（審核搶購報名與裁決申訴），以及 A3 的出貨、退款審核、結算對帳單、報名搶購、管理帳號，第一版沒有對應的 API。

S2、S3 不在第一版範圍。

5.5 沒有對應故事的規則與功能

部分規則是在設計階段才定下來的，或屬於第一版流程中沒有寫成故事的功能，同樣需要測試：

規則	出處	層級
存取權杖 15 分鐘後失效	ADR_009	API（以可控時鐘前進時間）
更新權杖換發後，舊的立即失效	ADR_009	API
已換發過的更新權杖再次被使用時，撤銷該帳號所有更新權杖	設計_004 3	API
登出、修改密碼後撤銷所有更新權杖	設計_005 2.7	API
更新權杖的 Cookie 帶有 `HttpOnly`、`Secure`、`SameSite=Strict`，`Path` 限定在對應前綴的 `/auth`	設計_005 2.7	API
以初始密碼登入的帳號，修改密碼前呼叫其他 API 回應 `PASSWORD_CHANGE_REQUIRED`	設計_005 2.7	API
帳號類型與路徑前綴不符時拒絕，例如會員的權杖呼叫賣家的 API	設計_005 2.1	API
錯誤回應符合 Problem Details，並帶有錯誤代碼與 trace ID	設計_005 2.4	API
冪等鍵的四種回應，包含處理中的 `IDEMPOTENCY_KEY_IN_USE`；結帳與發起付款都要驗證	設計_005 2.6	API、併發
冪等鍵以會員為範圍，不同會員使用相同的值互不影響	設計_005 2.6	API
不支援的每頁筆數、超過 1 年的日期區間回應 400；未指定區間時預設 30 天	設計_005 2.5	API
總筆數超過上限時 `totalItemsCapped` 為 true	設計_005 2.5	模組
回呼簽章錯誤、時間戳記超過 5 分鐘時拒絕	設計_005 3.5	API
回呼的商家交易編號不存在時仍回應 200，並記錄警告	設計_005 3.5	API
連線金流失敗時重試，重試送出相同的商家交易編號	設計_005 4.4	API
每位會員的預設收件資訊最多一筆；刪除後可再設定另一筆為預設	設計_004 3	模組
入庫與調整庫存：數字變化與異動紀錄（含原因）；調整後小於零時回應 `INVALID_STOCK_ADJUSTMENT` 且庫存不變	設計_005 3.3	API
購物車：加入、修改數量（1 到 99，會員自行輸入 100 件被拒絕）、移除；顯示目前價格、庫存狀態、是否已下架與是否超過結帳上限	需求 3.11、設計_005 3.2	API
庫存異動紀錄、狀態歷程、稽核紀錄帶有 trace ID，且與該次請求的 trace ID 相同	需求第 6 節	API
第二級稽核紀錄：設定抽成比例、變更營運帳號角色、發布條款、帳號鎖定與解鎖，記下操作者與變更前後的值；電話與地址遮罩	需求 3.6	API
稽核紀錄與業務資料在同一交易：業務交易撤銷時，稽核紀錄也不存在	需求 3.6	模組

《設計_005_API規格》2.4 的每個錯誤代碼，都至少要有一個測試觸發它，包含沒有對應故事的 `USERNAME_TAKEN`、`TERMS_NOT_LATEST`、`COMMISSION_NOT_APPLICABLE`。

請求頻率上限（429、`RATE_LIMITED`）在搶購階段實作，屆時補上測試。

6. 特殊的測試

6.1 併發測試

做法：同時啟動多個執行緒，以同一時間點放行，對同一筆資料送出請求，結束後驗證結果。

資料庫連線池的大小必須不小於執行緒數，否則請求會在連線池前排隊，實際上沒有同時發生，測試永遠通過。

第一版必須有的併發測試：

測試	驗證
同一冪等鍵同時結帳	只有一張結帳單；另一個請求回應 200 與同一張結帳單，或 409 `IDEMPOTENCY_KEY_IN_USE`，不會是 `PENDING_CHECKOUT_EXISTS`（見 5.1）
同一會員同時送出兩筆不同的結帳	只有一筆成功，另一筆為 `PENDING_CHECKOUT_EXISTS`
多人同時購買庫存只剩少量的商品	成功的數量不超過庫存；庫存不為負；異動紀錄加總與庫存數字一致
兩筆結帳以相反順序包含相同的兩項商品	兩筆都成功，沒有任何交易因死鎖被資料庫中止（《設計_004_資料庫設計》12.2）
同一結帳單同時發起付款	只有一筆處理中的付款
同一金流事件同時送達	付款狀態只更新一次，分錄只有一筆
取消與發起付款同時發生（會員主動取消，或逾時取消任務在付款期限前一刻開始的付款請求尚未寫入時執行）	只會出現三種結果之一：取消被拒絕、付款被拒絕，或兩者都成功。兩者都成功且付款之後成功時，結帳單維持已取消並發布「要求自動退款」事件（P4、ADR_013）。任何結果下，庫存都不會同時處於「已釋放」與「已確認扣除」
同一批次任務同時被觸發兩次	只有一次實際執行，只留下一筆執行紀錄（見 6.4）

併發測試的結果可能因執行時機而不同，每個測試重複執行 20 次，任何一次失敗都視為失敗。

6.2 模組邊界與程式慣例

檢查	工具
模組之間沒有違規依賴與循環依賴	Spring Modulith
業務程式不直接取得系統時間（4.1）	ArchUnit
SQL 與 Flyway 腳本不使用資料庫的時間函式（4.1）	掃描檔案的測試
業務程式不使用 `float`、`double` 型別，避免金額與比例出現小數誤差	ArchUnit
模組不存取其他模組的 `internal` 套件	Spring Modulith

6.3 Outbox 機制

驗證 ADR_007 要求的行為，而不只是相信函式庫：

測試	驗證
訂閱者處理失敗	事件維持未完成，之後被重新發送並成功處理
多個訂閱者其中一個失敗	只有失敗的訂閱者收到重送；成功的訂閱者不會重複收到
同一事件送達兩次	訂閱者只處理一次
業務交易撤銷	事件不會被發送
應用程式重啟	重啟前未完成的事件，重啟後會被發送

為了測試「付款已成功但訂單尚未更新」這類情境，測試中需要能暫停事件的發送，在指定時機再放行。做法於實作時決定，可能的方向：

- 測試專用的事件處理執行緒池，可由測試阻擋與放行
- 在測試中暫時停用訂閱者，讓事件維持未完成，之後再透過 Spring Modulith 的 `IncompleteEventPublications` 重新發送

不論採用哪一種，都以 6.3 的第一個測試確認暫停期間事件確實沒有被處理。

6.4 批次任務

第一版的批次任務是逾時取消與付款結果查詢，兩者都要驗證需求 3.5 的共通要求：

測試	驗證	層級
單一執行個體	同時觸發兩次，只有一次實際執行	併發
執行紀錄	留下任務名稱、資料範圍、起訖時間、成功與失敗筆數、最終結果	模組
單筆失敗	以替身讓其中一筆處理失敗，其他筆仍被處理；失敗筆數為 1，並記錄原因	模組
重複執行	對同一批資料再執行一次，結果不變，不產生重複的狀態歷程與異動紀錄	模組

執行中斷後續跑與手動觸發重跑，在每日任務的階段補上。

6.5 模擬金流的情境

API 測試以 HTTP 替身重現，端對端測試以真的模擬金流與付款代碼重現：

情境	API 測試的替身設定	端對端測試的付款代碼
成功	回應成功	`tok_success`
失敗	回應失敗	`tok_fail`
非同步成功	回應處理中，之後由測試送出回呼	`tok_async`
非同步失敗	回應處理中，之後由測試送出失敗的回呼	`tok_async_fail`

情境	API 測試的替身設定	端對端測試的付款代碼
逾時	回應延遲超過平台的回應逾時（測試環境已調短，見 4.5）	`tok_timeout`
回呼簽章錯誤	由測試送出簽章錯誤的回呼	不適用
回呼時間戳記過期	由測試送出 6 分鐘前的回呼	不適用
重複回呼	由測試送出相同事件編號兩次	開啟重複回呼的異常行為設定
連線失敗	第一次連線失敗、第二次回應成功	不適用

6.6 端對端測試

只涵蓋最關鍵的路徑，每條都同時經過兩個服務：

1. 註冊、登入、加入購物車、結帳、以 `tok_success` 付款，結帳單變為已付款
2. 以 `tok_timeout` 付款，付款先為處理中，之後經由回呼變為成功
3. 以 `tok_async_fail` 付款，回呼確認失敗後，以 `tok_success` 重新付款，結帳單變為已付款

端對端測試使用真實的時間，不涵蓋需要等待付款期限的情境；逾時取消與 P4 由 API、模組與併發測試驗證。

端對端測試需要先把兩個服務建置成容器映像檔，再以 Docker Compose 啟動。

6.7 模擬金流本身的測試

模擬金流是獨立的服務，有自己的單元與 API 測試：每個付款代碼的行為、以商家交易編號去除重複請求、回呼的簽章與重送、異常行為設定。

7. 覆蓋率

- 以 JaCoCo 產生報告，不設定讓建置失敗的強制門檻
- 核心模組（訂單、庫存、付款、帳本）的分支覆蓋率目標為 80%
- 由規格檔產生的程式不計入

覆蓋率只能看出哪些程式沒被執行過，無法看出測試是否驗證了正確的事。第 5 節的對應表才是判斷測試是否足夠的主要依據。

8. 完成的定義

一個功能符合以下全部條件才算完成：

1. 第 5 節對應表中該功能的所有情境都有測試且通過
2. `./mvnw verify` 全部通過，包含所有層級的測試、模組邊界驗證與程式格式檢查
3. API 有變動時，OpenAPI 規格檔已先更新，且 API 測試以規格檔驗證通過；新增的錯誤代碼已加入《設計_005_API規格》2.4，並有觸發它的測試
4. 資料表有變動時，《設計_004_資料庫設計》與 Flyway 遷移腳本已同步更新

5. 做了新的架構決策時，已寫成 ADR
6. 新增的批次任務、事件、狀態轉換，已在相關設計文件中補充

9. 執行方式

指令	執行的測試	時機
`./mvnw test`	單元、模組、架構測試	開發時頻繁執行
`./mvnw verify`	上述測試，加上 API、併發測試與程式格式檢查；不含端對端測試	提交程式之前
`./mvnw verify -Pe2e`	上述全部，加上端對端測試；會建置映像檔並啟動 Docker Compose	合併到主要分支之前

日後建立持續整合時，每次推送程式都執行 `./mvnw verify`。

10. 版本紀錄

版本	日期	說明
1.0	2026-10-14	初版定版，涵蓋第一版範圍
1.1	2026-10-15	檢視後修正：P4 改以模組與併發測試驗證，端對端路徑 3 改為失敗後重新付款；新增取消與發起付款同時發生的併發測試；O3 同時送出的預期回應（配合需求_002 1.6）；測試類別後綴與 Surefire、Failsafe 的分工；A2、A3 限定為第一版已有 API 的範圍；新增非同步事件與排程的做法、批次任務的測試；補充賣家條款、庫存、購物車、trace ID、稽核紀錄、冪等鍵範圍、錯誤代碼等遺漏的測試；時鐘只往前推進；O6 放回時合併數量的測試（配合需求_001 1.8）；引用 ADR_013